

PORTFOLIO PROJECT BUILD MANUAL

SQL & PySpark Hands-On

55 questions — logic, SQL and PySpark side by side

Fifty-five practice questions, each presented three ways: the logic to reason through first, then the SQL solution, then the equivalent PySpark. Designed to be worked through with the code covered.

This document contains

- 55 questions with paired SQL and PySpark solutions
- The reasoning stated before the code for each question
- Intended as active practice: cover the code, state the logic, then check

Vivek Pandey — Data Engineer

Azure · Databricks · PySpark · Delta Lake

vp171097.github.io/DevWebsite

55 interview questions. For each: the logic, then SQL, then the equivalent PySpark.

How to use this: cover the code, read only the logic, and try to write both yourself. The logic is the part that transfers between languages — once you have that, the syntax is just translation.

Sample schema used throughout (adjust names as needed):

```
employees(emp_id, name, dept, manager_id, salary, hire_date)
orders(order_id, customer_id, order_date, amount, status)
population_tbl(country_name, state, district, town, population)
logins(user_id, login_date)
```

Section A — Aggregation & Grouping

Q1. Total and average salary per department.

Logic: Straightforward GROUP BY. One row per department in the output.

```
SELECT dept, SUM(salary) AS total_salary, AVG(salary) AS avg_salary, COUNT(*) AS headcount
FROM employees
GROUP BY dept;
```

```
from pyspark.sql import functions as F

(emp.groupBy("dept")
 .agg(F.sum("salary").alias("total_salary"),
      F.avg("salary").alias("avg_salary"),
      F.count("*").alias("headcount")))
```

Q2. Departments with more than 10 employees.

Logic: Filter happens *after* aggregation, so it's HAVING, not WHERE. WHERE filters rows before grouping; HAVING filters groups after.

```
SELECT dept, COUNT(*) AS headcount
FROM employees
GROUP BY dept
HAVING COUNT(*) > 10;
```

```
(emp.groupBy("dept").agg(F.count("*").alias("headcount"))
 .filter(F.col("headcount") > 10))
```

Q3. Second highest salary — three ways.

Logic: Three different tools for the same job. Window functions handle ties explicitly and are the most maintainable; the subquery is fine for a single value; LIMIT/OFFSET is the quickest but least explicit about tie behaviour.

```
-- Window function (best – handles ties explicitly)
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees
) t WHERE rnk = 2;

-- Correlated-free subquery
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);

-- LIMIT/OFFSET
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

```
w = Window.orderBy(F.col("salary").desc())
(emp.withColumn("rnk", F.dense_rank().over(w))
  .filter("rnk = 2")
  .select("salary").distinct())
```

Q4. Count, but ignore soft-deleted rows and NULLs correctly.

Logic: `COUNT(*)` counts every row. `COUNT(col)` ignores NULLs in that column. These give different answers and interviewers use this to check you understand NULL semantics.

```
SELECT
  COUNT(*)           AS total_rows,
  COUNT(manager_id)  AS rows_with_manager,  -- NULLs excluded
  COUNT(DISTINCT dept) AS distinct_depts
FROM employees
WHERE is_deleted = false;
```

```
(emp.filter(~F.col("is_deleted")))
  .agg(F.count("*").alias("total_rows"),
       F.count("manager_id").alias("rows_with_manager"),
       F.countDistinct("dept").alias("distinct_depts"))
```

Q5. Top 3 highest-paid employees per department.

Logic: Partition by department, order by salary descending, keep rank ≤ 3 . This is the canonical "top-N per group" pattern — GROUP BY cannot do this because it collapses rows.

```
SELECT * FROM (
  SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk
  FROM employees e
) t WHERE rnk <= 3;

-- Databricks SQL shortcut:
SELECT * FROM employees
QUALIFY DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) <= 3;
```

```
w = Window.partitionBy("dept").orderBy(F.col("salary").desc())
(emp.withColumn("rnk", F.dense_rank().over(w))
  .filter("rnk <= 3")
  .drop("rnk"))
```

Section B — Joins

Q6. Employees with no manager assigned (self join).

Logic: A LEFT JOIN of the table to itself on `manager_id = emp_id`. Rows with no match get NULL on the right side.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

```
(emp.alias("e")
  .join(emp.alias("m"), F.col("e.manager_id") == F.col("m.emp_id"), "left")
  .select(F.col("e.name").alias("employee"), F.col("m.name").alias("manager")))
```

Q7. Customers who have never placed an order (anti join).

Logic: Want rows in the left table with **no** match in the right. Three equivalent ways to write it — **NOT EXISTS** / **LEFT ANTI JOIN** are NULL-safe; **NOT IN** is not.

```
-- Safe
SELECT c.* FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

-- Same result, join syntax
SELECT c.* FROM customers c
LEFT ANTI JOIN orders o ON c.customer_id = o.customer_id;

-- DANGEROUS if orders.customer_id can contain NULL – returns ZERO rows if so
SELECT * FROM customers WHERE customer_id NOT IN (SELECT customer_id FROM orders);
```

```
customers.join(orders, "customer_id", "left_anti")
```

Why NOT IN breaks: `x <> NULL` evaluates to UNKNOWN, not true or false, so if the subquery returns even one NULL, the whole NOT IN comparison can never be true for any row. Always prefer NOT EXISTS or LEFT ANTI JOIN.

Q8. Customers who HAVE placed at least one order (semi join).

Logic: The mirror of Q7 — matches in the right table, but you only want left-table columns, no duplication if a customer has multiple orders.

```
SELECT c.* FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

```
customers.join(orders, "customer_id", "left_semi")
```

Why not a regular join: An inner join on customer_id would duplicate each customer once per order. Semi join returns each matching customer exactly once, which is what "has placed an order" actually means.

Q9. Full outer join — reconcile two datasets (e.g. source vs target counts).

Logic: Keep everything from both sides, matched where possible, NULL where not. Used constantly for reconciliation: rows only in source are unmatched inserts, rows only in target are orphans.

```
SELECT COALESCE(s.id, t.id) AS id, s.amount AS source_amt, t.amount AS target_amt,
       CASE WHEN s.id IS NULL THEN 'only_in_target'
            WHEN t.id IS NULL THEN 'only_in_source'
            WHEN s.amount != t.amount THEN 'mismatch'
            ELSE 'match' END AS status
FROM source_tbl s
FULL OUTER JOIN target_tbl t ON s.id = t.id;
```

```
(source.alias("s").join(target.alias("t"), "id", "full_outer")
  .withColumn("status",
    F.when(F.col("s.id").isNull(), "only_in_target")
    .when(F.col("t.id").isNull(), "only_in_source")
    .when(F.col("s.amount") != F.col("t.amount"), "mismatch")
    .otherwise("match")))
```

Q10. Join condition on a range (e.g. assign each transaction to a pricing tier).

Logic: Not every join is on equality. A BETWEEN in the join condition matches a transaction to whichever tier range it falls into.

```
SELECT t.txn_id, t.amount, p.tier_name
FROM transactions t
JOIN pricing_tiers p ON t.amount BETWEEN p.min_amount AND p.max_amount;
```

```
txn.join(tiers,
  (txn.amount >= tiers.min_amount) & (txn.amount <= tiers.max_amount), "inner")
```

Caution: range joins don't benefit from a broadcast hash join structure the same way equality joins do — if `pricing_tiers` is small, explicitly broadcast it (`F.broadcast(tiers)`) so Spark doesn't attempt a sort-merge/cartesian-like plan.

Q11. Broadcast join — a large fact table joined to a small dimension.

Logic: When one side is small enough to fit in executor memory, Spark can ship it to every executor and avoid a shuffle entirely.

```
SELECT /*+ BROADCAST(d) */ f.*, d.country_name
FROM fact_sales f
JOIN dim_country d ON f.country_id = d.country_id;
```

```
from pyspark.sql.functions import broadcast

fact_sales.join(broadcast(dim_country), "country_id", "left")
```

Why it matters: without the hint, Spark decides automatically based on `autoBroadcastJoinThreshold` (default 10 MB) and the table's statistics. If statistics are stale (common after heavy appends without OPTIMIZE), Spark may pick a shuffle join for a table that's actually small — the explicit hint fixes that.

Section C — Window Functions

Q12. District-level population, all columns retained.

Logic: GROUP BY collapses the `town` column because it's not in the grouping key. A window function computes the aggregate but keeps every row.

```
SELECT country_name, state, district, town, population,
       SUM(population) OVER (PARTITION BY country_name, state, district) AS district_population
FROM population_tbl;
```

```
w = Window.partitionBy("country_name", "state", "district")
population_tbl.withColumn("district_population", F.sum("population").over(w))
```

Key point: no `ORDER BY` inside the window — that's what makes the frame the *entire partition* rather than a running total. Adding an ORDER BY here would silently change the answer.

Q13. RANK, DENSE_RANK, ROW_NUMBER — the difference.

Logic: They only differ on ties. Salaries 100, 90, 90, 80:

salary	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
90	2	2	2
90	3	2	2
80	4	4 (gap)	3 (no gap)

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,
       RANK()      OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk
FROM employees;
```

```
w = Window.orderBy(F.col("salary").desc())
(employeees
  .withColumn("rn", F.row_number().over(w))
  .withColumn("rnk", F.rank().over(w))
  .withColumn("drnk", F.dense_rank().over(w)))
```

When to use which: ROW_NUMBER for deduplication (you want exactly one row per key, ties broken arbitrarily unless you add a tiebreaker). DENSE_RANK for "top N per group" when ties should all be included.

Q14. Running total of daily revenue.

Logic: Order by date, sum from the start of the partition up to and including the current row. **ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** is explicit; without a frame clause the default with ORDER BY does the same thing but it's better to write it explicitly.

```
SELECT order_date, daily_revenue,
       SUM(daily_revenue) OVER (ORDER BY order_date
                               ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM daily_sales;
```

```
w = Window.orderBy("order_date").rowsBetween(Window.unboundedPreceding, Window.currentRow)
daily_sales.withColumn("running_total", F.sum("daily_revenue").over(w))
```

Q15. 3-day moving average.

Logic: A sliding frame of the current row plus the two preceding rows — **ROWS BETWEEN 2 PRECEDING AND CURRENT ROW**.

```
SELECT order_date, daily_revenue,
       AVG(daily_revenue) OVER (ORDER BY order_date
                               ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moving_avg_3d
FROM daily_sales;
```

```
w = Window.orderBy("order_date").rowsBetween(-2, 0)
daily_sales.withColumn("moving_avg_3d", F.avg("daily_revenue").over(w))
```

Q16. Month-over-month percentage change (LAG).

Logic: LAG looks one row back within the ordered partition. Guard the denominator with NULLIF to avoid divide-by-zero.

```
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
            / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 2) AS pct_change
FROM monthly_revenue;
```

```
w = Window.orderBy("month")
prev = F.lag("revenue").over(w)
(monthly_revenue
  .withColumn("prev_month_revenue", prev)
  .withColumn("pct_change",
    F.round(100.0 * (F.col("revenue") - prev) / F.nullif(prev, F.lit(0)), 2)))
```

Q17. First and last value in a partition (e.g. first login, most recent order status).

Logic: FIRST_VALUE / LAST_VALUE need an explicit frame — without one, LAST_VALUE only sees up to the current row and gives a different answer than expected.

```
SELECT user_id, order_date, status,
       FIRST_VALUE(status) OVER (
         PARTITION BY user_id ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS first_status,
       LAST_VALUE(status) OVER (
         PARTITION BY user_id ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_status
FROM orders;
```

```
w = Window.partitionBy("user_id").orderBy("order_date") \
  .rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing)
(orders
  .withColumn("first_status", F.first("status").over(w))
  .withColumn("last_status", F.last("status").over(w)))
```

The trap: without the explicit full-partition frame, **LAST_VALUE** uses the default running frame (**UNBOUNDED PRECEDING TO CURRENT ROW**), so it just returns the current row's own value — not the actual last row of the partition. This is one of the most common window-function bugs.

Q18. Percentile / NTILE — split customers into quartiles by spend.

Logic: NTILE(4) divides the ordered partition into four roughly equal buckets — a quick way to build quartile segments without computing exact percentile boundaries.

```
SELECT customer_id, total_spend,
       NTILE(4) OVER (ORDER BY total_spend DESC) AS spend_quartile
FROM customer_totals;
```

```
w = Window.orderBy(F.col("total_spend").desc())
customer_totals.withColumn("spend_quartile", F.ntile(4).over(w))
```

Q19. Percent of total (each row's share of the group sum).

Logic: Combine a partition-level sum window with the row's own value.

```
SELECT dept, name, salary,
       ROUND(100.0 * salary / SUM(salary) OVER (PARTITION BY dept), 2) AS pct_of_dept
FROM employees;
```

```
w = Window.partitionBy("dept")
employees.withColumn("pct_of_dept",
                    F.round(100.0 * F.col("salary") / F.sum("salary").over(w), 2))
```

Q20. Detect a value change from the previous row (state transition detection).

Logic: Compare each row's status to the previous row's status for the same entity. Where they differ, something changed — useful for building an audit trail from a snapshot table.

```
SELECT *,
       CASE WHEN status != LAG(status) OVER (PARTITION BY order_id ORDER BY updated_at)
            THEN 1 ELSE 0 END AS status_changed
FROM order_status_log;
```

```
w = Window.partitionBy("order_id").orderBy("updated_at")
(order_status_log
 .withColumn("prev_status", F.lag("status").over(w))
 .withColumn("status_changed",
             F.when(F.col("status") != F.col("prev_status"), 1).otherwise(0)))
```

Section D — Deduplication

Q21. Keep only the latest record per key.

Logic: Window partitioned by the business key, ordered by recency descending, keep rank 1. Always add a tiebreaker column (like ingest timestamp) so the result is deterministic when two records share the exact same update time.

```
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY id ORDER BY updated_at DESC, ingest_ts DESC) AS rn
  FROM raw_events
) t WHERE rn = 1;

-- Databricks SQL shortcut
SELECT * FROM raw_events
QUALIFY ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC, ingest_ts DESC) = 1;
```

```
w = Window.partitionBy("id").orderBy(F.col("updated_at").desc(), F.col("ingest_ts").desc())
(raw_events.withColumn("rn", F.row_number().over(w))
 .filter("rn = 1").drop("rn"))
```

Q22. Find duplicate rows (exact duplicates, not just same key).

Logic: Group by every column, and any group with more than one row is a duplicate.

```
SELECT id, name, email, COUNT(*) AS cnt
FROM customers
GROUP BY id, name, email
HAVING COUNT(*) > 1;
```

```
(customers.groupBy("id", "name", "email")
  .count()
  .filter("count > 1"))
```

Q23. Delete duplicates in place, keeping one row (Delta).

Logic: Identify the rows to remove — everything except rank 1 per key — then delete exactly those. On Delta this is most safely done as a MERGE or a full rewrite rather than a row-by-row DELETE.

```
-- Rewrite approach (simplest, safe for a batch cleanup)
CREATE OR REPLACE TABLE customers AS
SELECT * EXCEPT (rn) FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) AS rn
  FROM customers
) WHERE rn = 1;
```

```
w = Window.partitionBy("id").orderBy(F.col("updated_at").desc())
deduped = (spark.table("cat.sch.customers")
  .withColumn("rn", F.row_number().over(w))
  .filter("rn = 1").drop("rn"))

deduped.write.format("delta").mode("overwrite").saveAsTable("cat.sch.customers")
```

Q24. Streaming deduplication with a bounded watermark.

Logic: In a stream, `dropDuplicates` alone keeps every key ever seen — unbounded state. Adding a watermark lets the dedup window close and evict old keys.

```
(events
  .withWatermark("event_ts", "30 minutes")
  .dropDuplicatesWithinWatermark(["event_id"]))
```

There's no SQL streaming equivalent commonly used outside DLT; in DLT the same intent is expressed with `APPLY CHANGES INTO ... SEQUENCE BY` and dropping already-seen keys structurally.

Section E — Gaps, Islands & Sessionization

Q25. Consecutive login streaks (gaps and islands).

Logic: The classic trick — subtract a row number from the date. For a consecutive run of dates, `date - row_number` is constant, so grouping by that constant identifies each streak.

```
WITH grouped AS (  
  SELECT user_id, login_date,  
         DATE_SUB(login_date, ROW_NUMBER() OVER (  
           PARTITION BY user_id ORDER BY login_date)) AS grp  
  FROM (SELECT DISTINCT user_id, login_date FROM logins)  
)  
SELECT user_id, MIN(login_date) AS streak_start, MAX(login_date) AS streak_end,  
       COUNT(*) AS streak_length  
FROM grouped  
GROUP BY user_id, grp  
HAVING COUNT(*) >= 3;
```

```
w = Window.partitionBy("user_id").orderBy("login_date")  
grouped = (logins.dropDuplicates(["user_id", "login_date"])  
           .withColumn("rn", F.row_number().over(w))  
           .withColumn("grp", F.date_sub(F.col("login_date"), F.col("rn"))))  
  
(grouped.groupBy("user_id", "grp")  
  .agg(F.min("login_date").alias("streak_start"),  
       F.max("login_date").alias("streak_end"),  
       F.count("*").alias("streak_length"))  
  .filter("streak_length >= 3"))
```

Q26. Sessionize events (new session after 30 minutes of inactivity).

Logic: Compare each event's timestamp to the previous event for the same user. If the gap exceeds the threshold, it's a new session — flag it with 1, otherwise 0, then a running sum of those flags gives a session ID.

```
WITH gaps AS (  
  SELECT user_id, event_ts,  
         CASE WHEN event_ts > LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts)  
              + INTERVAL 30 MINUTES  
         THEN 1 ELSE 0 END AS new_session_flag  
  FROM events  
)  
SELECT *,  
       SUM(new_session_flag) OVER (PARTITION BY user_id ORDER BY event_ts  
                                   ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS session_id  
FROM gaps;
```

```
w = Window.partitionBy("user_id").orderBy("event_ts")
gaps = (events
  .withColumn("prev_ts", F.lag("event_ts").over(w))
  .withColumn("gap_minutes",
    (F.col("event_ts").cast("long") - F.col("prev_ts").cast("long")) / 60)
  .withColumn("new_session_flag",
    F.when((F.col("gap_minutes") > 30) | F.col("prev_ts").isNull(), 1).otherwise(0)))

w_running = w.rowsBetween(Window.unboundedPreceding, Window.currentRow)
gaps.withColumn("session_id", F.sum("new_session_flag").over(w_running))
```

For a real streaming version of this, use `flatMapGroupsWithState` or `transformWithState` so the session boundary is computed incrementally with a timeout, rather than requiring the whole history in memory.

Q27. Customers active in every month of a given year.

Logic: Group by customer, count distinct months they ordered in, and check it equals 12.

```
SELECT customer_id
FROM orders
WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01'
GROUP BY customer_id
HAVING COUNT(DISTINCT MONTH(order_date)) = 12;
```

```
(orders.filter((F.col("order_date") >= "2024-01-01") & (F.col("order_date") < "2025-01-01")))
  .groupBy("customer_id")
  .agg(F.countDistinct(F.month("order_date")).alias("active_months"))
  .filter("active_months = 12"))
```

Section F — Pivot, Unpivot, Grouping Sets

Q28. Pivot — quarterly revenue as columns.

Logic: Turn distinct row values (quarters) into columns, aggregating into the intersection. Always list the pivot values explicitly in PySpark — otherwise Spark runs an extra job just to discover them.

```
SELECT * FROM (SELECT region, quarter, revenue FROM sales)
PIVOT (SUM(revenue) FOR quarter IN ('Q1','Q2','Q3','Q4'));
```

```
(sales.groupBy("region")
  .pivot("quarter", ["Q1", "Q2", "Q3", "Q4"]) # explicit list avoids a discovery job
  .sum("revenue"))
```

Q29. Unpivot — columns back to rows.

Logic: The inverse of pivot — several wide columns collapse into key/value row pairs. SQL uses

UNPIVOT (or a UNION ALL if the dialect lacks it); PySpark uses `stack`.

```
SELECT region, quarter, revenue
FROM sales_wide
UNPIVOT (revenue FOR quarter IN (Q1, Q2, Q3, Q4));
```

```
sales_wide.selectExpr(
    "region",
    "stack(4, 'Q1', Q1, 'Q2', Q2, 'Q3', Q3, 'Q4', Q4) as (quarter, revenue)")
```

Q30. Subtotals and grand total in one query (ROLLUP).

Logic: ROLLUP produces the normal grouped rows, plus subtotal rows with NULL in the finer grouping columns, plus a grand total row with NULL in all of them.

```
SELECT country, state, SUM(population) AS total_population
FROM population_tbl
GROUP BY ROLLUP(country, state);
-- Rows: (country,state) pairs, then (country, NULL) subtotals, then (NULL,NULL) grand total
```

```
(population_tbl.rollup("country_name", "state")
    .agg(F.sum("population").alias("total_population"))
    .orderBy("country_name", "state"))
```

Section G — Hierarchies & Recursion

Q31. Employee-manager hierarchy with level depth (recursive CTE).

Logic: Start from the root (no manager), then repeatedly join employees to the previous level's results, incrementing depth, until no more matches.

```
WITH RECURSIVE org AS (
    SELECT emp_id, name, manager_id, 1 AS lvl
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.name, e.manager_id, o.lvl + 1
    FROM employees e
    JOIN org o ON e.manager_id = o.emp_id
)
SELECT * FROM org ORDER BY lvl;
```

```
# PySpark has no native recursive CTE – implement with an iterative loop
level = 1
current = employees.filter(F.col("manager_id").isNull()) \
    .withColumn("lvl", F.lit(level))
result = current

while current.count() > 0:
    level += 1
    current = (employees.alias("e")
        .join(current.alias("o"), F.col("e.manager_id") == F.col("o.emp_id"))
        .select("e.emp_id", "e.name", "e.manager_id", (F.col("o.lvl") + 1).alias("lvl")))
    result = result.unionByName(current)

result.orderBy("lvl")
```

Why the loop: Spark SQL on Databricks does support `WITH RECURSIVE` directly (as shown above) — but the DataFrame API has no built-in recursive primitive, so if you're doing this in pure PySpark rather than `spark.sql()`, the iterative join-and-union loop is the standard workaround. Cap it with a max-iterations guard in production to avoid an infinite loop on cyclic data.

Q32. Find the top-level ancestor for every employee.

Logic: Walk the hierarchy upward instead of downward — same recursive shape, just starting from each row and following `manager_id` until it's NULL.

```
WITH RECURSIVE chain AS (
    SELECT emp_id, name, manager_id, emp_id AS original_emp, name AS path
    FROM employees
    UNION ALL
    SELECT e.emp_id, e.name, m.manager_id, c.original_emp, c.path || ' -> ' || m.name
    FROM chain c
    JOIN employees e ON c.manager_id = e.emp_id
    JOIN employees m ON e.manager_id = m.emp_id
)
SELECT original_emp, MAX(path) AS full_chain FROM chain GROUP BY original_emp;
```

This one is usually left as a talking point rather than written live — explain the walk-upward logic and reference Q31's pattern rather than re-deriving the full query under time pressure.

Section H — Strings & Dates

Q33. Extract domain from an email column.

Logic: Split on `@` and take the second part. `split` returns an array; index into it.

```
SELECT email, SPLIT(email, '@')[1] AS domain
FROM customers;
```

```
customers.withColumn("domain", F.split(F.col("email"), "@").getItem(1))
```

Q34. Standardize inconsistent date strings.

Logic: Different source systems often send dates in different formats. `to_date` with an explicit pattern is much safer than relying on inference, which silently produces NULL on a mismatch.

```
SELECT raw_date,
       COALESCE(TRY_TO_DATE(raw_date, 'yyyy-MM-dd'),
                TRY_TO_DATE(raw_date, 'dd/MM/yyyy'),
                TRY_TO_DATE(raw_date, 'MM-dd-yyyy')) AS clean_date
FROM raw_source;
```

```
from pyspark.sql.functions import coalesce, to_date

df.withColumn("clean_date",
              F.coalesce(F.to_date("raw_date", "yyyy-MM-dd"),
                         F.to_date("raw_date", "dd/MM/yyyy"),
                         F.to_date("raw_date", "MM-dd-yyyy")))
```

Q35. Age of a record in business-relevant buckets.

Logic: `datediff` for the raw day count, then a CASE/when chain to bucket it. Order the conditions from most specific to least — the first match wins.

```
SELECT order_id, order_date,
       DATEDIFF(current_date(), order_date) AS days_old,
       CASE WHEN DATEDIFF(current_date(), order_date) <= 7 THEN 'This week'
            WHEN DATEDIFF(current_date(), order_date) <= 30 THEN 'This month'
            WHEN DATEDIFF(current_date(), order_date) <= 365 THEN 'This year'
            ELSE 'Older' END AS age_bucket
FROM orders;
```

```
days_old = F.datediff(F.current_date(), F.col("order_date"))
(orders.withColumn("days_old", days_old)
 .withColumn("age_bucket",
             F.when(days_old <= 7, "This week")
             .when(days_old <= 30, "This month")
             .when(days_old <= 365, "This year")
             .otherwise("Older")))
```

Q36. Fiscal quarter from a date (non-standard fiscal year starting in April).

Logic: Shift the month before computing the quarter, so April becomes "month 1" of the fiscal year.

```
SELECT order_date,
       CONCAT('FY', YEAR(order_date) + CASE WHEN MONTH(order_date) >= 4 THEN 1 ELSE 0 END,
             '-Q', CEIL(((MONTH(order_date) + 8) % 12 + 1) / 3.0)) AS fiscal_quarter
FROM orders;
```

```
shifted_month = ((F.month("order_date") + 8) % 12) + 1      # April -> 1, March -> 12
fiscal_year = F.year("order_date") + F.when(F.month("order_date") >= 4, 1).otherwise(0)
orders.withColumn("fiscal_quarter",
                  F.concat(F.lit("FY"), fiscal_year, F.lit("-Q"), F.ceil(shifted_month / 3)))
```

This one's mostly a "walk me through your thinking" question rather than something you're expected to get byte-perfect live — the logic (shift the month, then quarter it) is what they're checking for.

Section I — Nested & Semi-Structured Data

Q37. Explode an array column (order items).

Logic: One order row with an array of items becomes one row per item. `explode` drops the row entirely if the array is NULL or empty — `explode_outer` keeps it with NULL item columns instead.

```
SELECT order_id, item.sku, item.qty
FROM orders LATERAL VIEW EXPLODE(items) t AS item;

-- keep orders with an empty items array
SELECT order_id, item.sku, item.qty
FROM orders LATERAL VIEW OUTER EXPLODE(items) t AS item;
```

```
orders.select("order_id", F.explode("items").alias("item")) \
      .select("order_id", "item.sku", "item.qty")

# outer version – keeps rows with an empty/null array
orders.select("order_id", F.explode_outer("items").alias("item"))
```

Q38. Explode with the array index (POSEXPLODE).

Logic: Same as Q37 but also keep the original position within the array — useful when order matters (e.g. line item sequence).

```
SELECT order_id, pos, item.sku
FROM orders LATERAL VIEW POSEXPLODE(items) t AS pos, item;
```

```
orders.select("order_id", F.posexplode("items").alias("pos", "item")) \
      .select("order_id", "pos", "item.sku")
```

Q39. Flatten an arbitrarily nested JSON struct (recursive flatten).

Logic: You don't know the nesting depth ahead of time, so loop: on each pass, expand every struct

column into prefixed top-level columns and explode every array column, and repeat until nothing nested remains.

```
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, ArrayType

def flatten(df):
    while True:
        structs = [f.name for f in df.schema.fields if isinstance(f.dataType, StructType)]
        arrays = [f.name for f in df.schema.fields if isinstance(f.dataType, ArrayType)]
        if not structs and not arrays:
            return df
        for c in structs:
            df = df.select("*, *[F.col(f'{c}.{sf.name}').alias(f'{c}_{sf.name}')
                           for sf in df.schema[c].dataType.fields]).drop(c)
        for c in arrays:
            df = df.withColumn(c, F.explode_outer(c))
```

No clean SQL equivalent — this is a case where you'd say so directly: "I'd handle this in PySpark with a loop, because SQL doesn't have a generic way to introspect and expand an unknown nesting depth."

Q40. Extract a value from JSON stored as a string column.

Logic: Three approaches depending on how much of the JSON you need. `get_json_object` for a single field, `from_json` with a schema when you need many fields, and colon notation on the `variant` type for the newest, most convenient option.

```
SELECT
  get_json_object(raw, '$.user.id') AS user_id,           -- one field, string path
  raw:user.name::string AS user_name,                   -- variant colon notation
  from_json(raw, 'STRUCT<id:INT, name:STRING>') AS parsed -- many fields at once
FROM bronze_events;
```

```
schema = "STRUCT<id: INT, name: STRING>"
(bronze_events
  .withColumn("user_id", F.get_json_object("raw", "$.user.id"))
  .withColumn("parsed", F.from_json("raw", schema)))
```

When to use which: `get_json_object` for one or two fields where defining a schema is overkill. `from_json` when you need most of the object — it's a single parse pass rather than one regex-like extraction per field.

Section J — Delta-Specific Operations

Q41. SCD Type 2 upsert.

Logic: A single MERGE can't both close an old row and insert a new one for the same key, because MERGE won't match the same key twice. The standard workaround stages every changed record twice: once with the real key (which matches the current row and closes it) and once with a null

merge key (which never matches, so it falls through to the insert branch).

```
from delta.tables import DeltaTable
from pyspark.sql import functions as F

tgt = DeltaTable.forName(spark, "cat.gold.dim_customer")
src = src.withColumn("hash", F.sha2(F.concat_ws("||", "name", "segment", "city"), 256))

changed = (src.alias("s")
    .join(tgt.toDF().filter("is_current").alias("t"), "cust_id", "left")
    .filter(F.col("t.cust_id").isNull() | (F.col("s.hash") != F.col("t.hash")))
    .select("s.*"))

staged = (changed.withColumn("merge_key", F.col("cust_id"))
    .unionByName(changed.withColumn("merge_key", F.lit(None).cast("string"))))

(tgt.alias("t")
    .merge(staged.alias("s"), "t.cust_id = s.merge_key AND t.is_current")
    .whenMatchedUpdate(condition="t.hash <> s.hash", set={
        "is_current": F.lit(False), "effective_to": F.col("s.effective_from")})
    .whenNotMatchedInsert(values={
        "cust_id": "s.cust_id", "name": "s.name", "hash": "s.hash",
        "effective_from": "s.effective_from",
        "effective_to": F.lit("9999-12-31").cast("date"), "is_current": F.lit(True)})
    .execute())

-- DLT collapses all of the above into one declaration
APPLY CHANGES INTO live.dim_customer
FROM STREAM(live.bronze_customer)
KEYS (cust_id) SEQUENCE BY seq_num
STORED AS SCD TYPE 2;
```

Q42. Idempotent streaming upsert (foreachBatch MERGE).

Logic: The streaming writer can only append, update, or complete — it can't MERGE. `foreachBatch` gives you the micro-batch as a plain DataFrame so you can run a Delta MERGE against it, which is what makes the write idempotent on replay.

```

from delta.tables import DeltaTable

def upsert(batch_df, batch_id):
    w = Window.partitionBy("id").orderBy(F.col("updated_at").desc())
    batch_df = (batch_df.withColumn("rn", F.row_number().over(w))
                .filter("rn = 1").drop("rn"))    # MERGE needs unique source keys

    (DeltaTable.forName(spark, "cat.silver.events").alias("t")
     .merge(batch_df.alias("s"), "t.id = s.id AND t.event_date = s.event_date")
     .whenMatchedUpdateAll(condition="s.updated_at > t.updated_at")
     .whenNotMatchedInsertAll()
     .execute())

(stream.writeStream
 .foreachBatch(upsert)
 .option("checkpointLocation", CHECKPOINT_PATH)
 .trigger(availableNow=True)
 .start())

```

No SQL equivalent — this is inherently a PySpark/Scala pattern because `foreachBatch` is a streaming API construct.

Q43. Retention: keep only the last five years of business data.

Logic: Two separate operations that are frequently confused. DELETE removes rows logically (business retention). VACUUM reclaims physical storage for files no longer referenced by the current version (it does not implement retention by itself).

```

-- 1. Business-level retention – this is what actually keeps 5 years of rows
DELETE FROM cat.silver.txn
WHERE event_date < add_months(current_date(), -60);

-- 2. Physical reclamation – controls file history / time-travel depth, NOT row retention
ALTER TABLE cat.silver.txn SET TBLPROPERTIES (delta.deletedFileRetentionDuration = 'interval 7 days');
VACUUM cat.silver.txn RETAIN 168 HOURS;    -- the unit is HOURS

```

```

from datetime import datetime, timedelta

cutoff = (datetime.now() - timedelta(days=5*365)).strftime("%Y-%m-%d")

spark.sql(f"""
    DELETE FROM cat.silver.txn WHERE event_date < '{cutoff}'
""")

spark.sql("VACUUM cat.silver.txn RETAIN 168 HOURS")

```

Q44. Reprocess a single day without touching the rest of the table.

Logic: `replaceWhere` atomically overwrites only the rows matching the predicate — needed constantly for reruns and backfills where a full table overwrite would be wasteful and dangerous.

```
INSERT OVERWRITE TABLE cat.silver.invoice
REPLACE WHERE event_date = '2026-08-18'
SELECT * FROM staged_invoice_20260818;
```

```
(rebuilt_day.write.format("delta").mode("overwrite")
    .option("replaceWhere", "event_date = '2026-08-18'")
    .saveAsTable("cat.silver.invoice"))
```

Q45. Read Change Data Feed for incremental downstream processing.

Logic: CDF exposes row-level inserts, updates, and deletes as a queryable stream of changes, instead of forcing consumers to diff full snapshots.

```
ALTER TABLE cat.silver.invoice SET TBLPROPERTIES (delta.enableChangeDataFeed = true);

SELECT *, _change_type, _commit_version
FROM table_changes('cat.silver.invoice', 120, 135);
```

```
(spark.readStream.format("delta")
    .option("readChangeFeed", "true")
    .option("startingVersion", 120)
    .table("cat.silver.invoice")
    .filter(F.col("_change_type").isin("insert", "update_postimage")))
```

Section K — PySpark-Specific Operations

Q46. Handle a corrupt/malformed record without failing the job.

Logic: `PERMISSIVE` mode (the default) puts unparseable rows into a designated column instead of throwing, so the job continues and you can inspect the bad rows separately. No direct SQL equivalent — this is a read-time option.

```

from pyspark.sql.types import StructType, StructField, StringType, IntegerType

schema = (StructType([
    StructField("id", IntegerType()), StructField("name", StringType())])
    .add("_corrupt_record", StringType()))

df = (spark.read.schema(schema)
    .option("mode", "PERMISSIVE")
    .option("columnNameOfCorruptRecord", "_corrupt_record")
    .json(path))

good = df.filter(F.col("_corrupt_record").isNull())
bad = df.filter(F.col("_corrupt_record").isNotNull())

```

Q47. Vectorized transformation with a pandas UDF (avoiding a slow row-at-a-time UDF).

Logic: A plain Python UDF serializes every row across the JVM-Python boundary individually. A pandas UDF operates on a whole batch (a pandas Series) at once via Arrow, which is typically an order of magnitude faster.

```

from pyspark.sql.functions import pandas_udf
import pandas as pd

@pandas_udf("double")
def z_score(s: pd.Series) -> pd.Series:
    return (s - s.mean()) / s.std()

df.withColumn("amount_z", z_score("amount"))

```

No SQL equivalent — but the honest interview answer is: "I'd try to express this with built-in functions first; a pandas UDF is for logic that genuinely can't be expressed natively, like a statistical transform or a call into a Python ML library."

Q48. Apply row-wise pandas logic at scale without collecting to the driver (mapInPandas).

Logic: `toPandas()` pulls the entire DataFrame to the driver and will OOM on large data. `mapInPandas` runs your pandas function per partition, on the executors, so it scales.

```

def score_partition(iterator):
    for pdf in iterator:
        pdf["score"] = pdf["a"] * 0.3 + pdf["b"] * 0.7
        yield pdf

df.mapInPandas(score_partition, schema="a double, b double, score double")

```

Q49. Union two DataFrames safely when column order might differ.

Logic: Plain `union` matches columns by **position**, not name — a silent, dangerous bug if the two DataFrames' columns are in a different order. `unionByName` matches by name explicitly.

```
-- SQL UNION matches by position too, so column order must match, or alias explicitly
SELECT id, name, amount FROM tbl_a
UNION ALL
SELECT id, name, amount FROM tbl_b;  -- must already be in the same column order
```

```
# WRONG if columns are in a different order – silently misaligns data
a.union(b)

# SAFE – matches by column name
a.unionByName(b, allowMissingColumns=True)
```

Q50. Repartition before writing to control output file count.

Logic: Without control, the number of output files equals the number of in-memory partitions, which is often far more than the data warrants. `repartition(col)` before a partitioned write gives you roughly one file per partition value instead of N.

```
(df.repartition("country")
  .write.format("delta")
  .partitionBy("country")
  .mode("overwrite")
  .saveAsTable("cat.gold.sales_by_country"))
```

No SQL equivalent — file layout control at write time is a DataFrame-writer-level concern, though `OPTIMIZE` afterwards achieves a similar end state for an existing Delta table.

Q51. Broadcast a Python lookup dictionary (not a DataFrame) into a UDF.

Logic: When the lookup is small and truly static (not a DataFrame you'd otherwise query), broadcasting a plain Python object avoids re-sending it with every task closure.

```
lookup = {"US": "North America", "IN": "Asia", "DE": "Europe"}
bc_lookup = spark.sparkContext.broadcast(lookup)

@F.udf("string")
def region_for(country_code):
    return bc_lookup.value.get(country_code, "Unknown")

df.withColumn("region", region_for("country_code"))
```

Better alternative when possible: if the lookup can be expressed as a small DataFrame, prefer a real broadcast join (Q11) over a UDF — it stays visible to Catalyst and avoids the Python UDF overhead entirely.

Q52. Read and process only newly arrived files incrementally (Auto Loader).

Logic: Rather than re-scanning a whole directory, Auto Loader tracks which files it has already processed and only picks up new ones — essential at high file counts. No SQL equivalent; this is a DataFrame reader configuration.

```
bronze = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", SCHEMA_LOC)
    .option("cloudFiles.schemaEvolutionMode", "rescue")
    .option("cloudFiles.maxFilesPerTrigger", 20000)
    .load(LANDING_PATH))

(bronze.writeStream
    .option("checkpointLocation", CHECKPOINT_PATH)
    .trigger(availableNow=True)
    .toTable("cat.bronze.events"))
```

Section L — More SQL Fundamentals

Q53. QUALIFY — filter on a window function result without a subquery.

Logic: Without QUALIFY, filtering on a window function result requires wrapping the query in a subquery just to apply the WHERE. QUALIFY does it directly, which is cleaner and is supported in Databricks SQL.

```
-- without QUALIFY
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
    FROM products
) WHERE rn <= 3;

-- with QUALIFY – same result, no subquery
SELECT * FROM products
QUALIFY ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) <= 3;
```

```
# PySpark has no QUALIFY keyword – you write the equivalent subquery pattern
w = Window.partitionBy("category").orderBy(F.col("revenue").desc())
(products.withColumn("rn", F.row_number().over(w))
    .filter("rn <= 3").drop("rn"))
```

Q54. GROUPING SETS — multiple grouping levels in one query.

Logic: ROLLUP gives you a fixed hierarchy of subtotals. GROUPING SETS lets you specify exactly which combinations you want, including ones that skip levels — e.g. total by country and separately total by state, without a joint country+state breakdown.

```
SELECT country, state, SUM(population) AS total_population
FROM population_tbl
GROUP BY GROUPING SETS ((country), (state), ());
```

```
by_country = population_tbl.groupBy("country_name").agg(F.sum("population").alias("total"))
by_state   = population_tbl.groupBy("state").agg(F.sum("population").alias("total"))
grand_total = population_tbl.agg(F.sum("population").alias("total"))
# combined with unionByName(allowMissingColumns=True) if a single result set is needed
```

Q55. CTE vs subquery vs temp view — does it matter for performance?

Logic: In Spark SQL, a CTE (`WITH ...`) is not materialized — it's purely syntactic. If you reference it three times in the outer query, it's recomputed three times, exactly like a subquery would be. A temp view is the same thing under a name. If you genuinely need the intermediate result computed once and reused, you have to force materialization explicitly.

```
-- referenced twice -> computed twice, CTE or not
WITH high_value AS (SELECT * FROM orders WHERE amount > 1000)
SELECT (SELECT COUNT(*) FROM high_value) AS cnt,
       (SELECT SUM(amount) FROM high_value) AS total;
```

```
# force materialization if it's genuinely reused and expensive
high_value = orders.filter("amount > 1000").cache()
cnt = high_value.count()
total = high_value.agg(F.sum("amount")).collect()[0][0]
high_value.unpersist()
```

The interview point: readability differs between CTE/subquery/view, but raw execution cost usually doesn't — that surprises people coming from traditional RDBMS backgrounds where CTEs can sometimes be materialized automatically.

Quick-Reference: Logic Patterns Cheat Sheet

Pattern	Recognize it by	Tool
Keep every row, add a group aggregate	"all columns displayed" alongside a total	Window function, no ORDER BY in the window
Top N per group	"top 3 per category/department"	<code>RANK</code> / <code>DENSE_RANK</code> + filter, or <code>QUALIFY</code>
Running/cumulative value	"running total", "cumulative"	Window with <code>ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW</code>
Compare to previous/next row	"month-over-month", "change from last"	<code>LAG</code> / <code>LEAD</code>
Rows in A with no match in B	"never ordered", "not in", "missing from"	<code>LEFT ANTI JOIN</code> / <code>NOT EXISTS</code> — never <code>NOT IN</code>
Rows in A with a match in B, no duplication	"has at least one"	<code>LEFT SEMI JOIN</code> / <code>EXISTS</code>
Consecutive sequences	"streak", "consecutive days"	$\text{date} - \text{row_number} = \text{constant group key}$
Reset counter on a condition	"session", "new session after X minutes"	flag + running <code>SUM</code> of the flag
One row per key, latest wins	"deduplicate", "keep the latest"	<code>ROW_NUMBER</code> partitioned by key, ordered by recency, filter $\text{rn}=1$
Columns to rows / rows to columns	"one row per X instead of a column per X"	<code>PIVOT</code> / <code>UNPIVOT</code> (SQL), <code>stack</code> (PySpark)
Unknown nesting depth	"flatten this JSON"	Loop expanding structs/arrays until none remain
Point-in-time history	"SCD", "track changes over time"	Type 2 two-sided MERGE staging, or <code>APPLY CHANGES INTO</code>